Empirical Software Engineering (2025) 30:56
https://doi.org/10.1007/s10664-024-10599-6

Developer reactions to protestware in open source software: the cases of `color.js` and `es5.ext`

Youmei Fan[1] · Dong Wang[2] · Supatsara Wattanakriengkrai[1] · Hathaichanok Damrongsiri[1] · Christoph Treude[3] · Hideaki Hata[4] · Raula Gaikovina Kula[5]

Accepted: 26 November 2024 / Published online: 18 January 2025
© The Author(s) 2025

Abstract
There is growing concern about maintainers self-sabotaging their work in order to take political or economic stances, a practice referred to as "protestware". Our objective is to understand the discourse around discussions on such an attack, how it is received by the community, and whether developers respond to the attack in a timely manner. We study two notable protestware cases i.e., colors.js and es5-ext. Results indicate that protestware discussions are spread more quickly on the GitHub platform, while security vulnerabilities are faster on social media. By establishing a taxonomy of protestware discussions, we identify posts that express stances and provide technical mitigation instructions. We applied a thematic analysis to 684 protestware related posts to identify five major themes during the discussions: i. disseminate and response, ii. stance, iii. reputation, iv. communicative styles, v. rights and ethics. This work sheds light on the nuanced landscape of protestware discussions, offering insights for both researchers and developers into maintaining a healthy balance between the political or social actions of developers and the collective well-being of the open-source community.

Keywords Protestware · Software ecosystems · Open source software

1 Introduction

> *"Anyone who experienced actual significant disruption from this (protestware) brought it on themselves with their bad dev practices. No one forced anyone to install the latest version without actually verifying it at all. Didnt corrupt the version history so nope, people just letting their entitlement and lack of understanding of licenses show."*

> *"In dev-land, we don't stand on the shoulders of giants. We keep our life-rafts afloat by sticky-taping together skerricks of code that hopefully has more buoyancy than ballast. And sometime it just takes one person to take the whole ship down."*

Communicated by: Fabio Calefato, Hourieh Khalajzadeh and Igor Steinmacher

This article is part of the Topical Collection: *Special Issue on CHASE 2023*

Extended author information available on the last page of the article

Open source software development has emerged as an unexpected platform for expressing social and political protests. In recent years, we have witnessed the emergence of "protestware", defined as when developers deliberately express dissent and draw attention to issues they consider important (Kula and Treude 2022). Protestware takes on many forms. For example, the maintainer of 'node-ipc' used malicious code to target host machines with IP addresses in Russia or Belarus in response to the War in Ukraine (Massacci et al. 2022). GitHub declared this a critical vulnerability, known as CVE-2022-23812.[1] This act underscores the political motivations that can inspire the creation of protestware. Other instances of protestware stem from industry concerns, such as the perceived exploitation of open-source labor by large corporations. For instance, the developer of the 'faker' library intentionally introduced an infinite loop, disrupting thousands of projects (Bellovin 2022). Today, 'faker' is a community-controlled project, maintained by a team of developers from various companies (Sachs and the Faker Team 2022).

Such protests can result in widespread consequences, particularly as modern software systems often heavily rely on third-party libraries, rendering them potential points of vulnerability (Zahan et al. 2022). To illustrate the scale of this interconnectedness, libraries listed in the popular NPM registry, which hosts over a million libraries, each depend on an average of five to six other libraries within the same ecosystem (Chinthanet et al. 2021). Beyond the technical issues, the rise of protestware threatens the trust that is foundational to modern software ecosystems (Ghofrani et al. 2022), creating disruptions within the software development community. Developers using a protesting library must decide whether to continue using it or take on the potentially complex task of finding alternatives (He et al. 2021) or reverting to older versions (Kula et al. 2018). Societal reactions to protestware vary, with responses ranging from strong support to harsh criticism, and everything in between (Massacci et al. 2022). Through the GitHub comments and social media, we can also gain insights into the practicality of hosting protestware, and in what instances should protestware become tolerated. This, in turn, contributes to the creation of a more friendly software ecosystem.

In this paper, we study the disruptions caused by protestware and the immediate reactions it prompts. Specifically, we explore the recent cases colors.js and es5-ext as instances of protestware. We utilize ua-parser and log4j as baselines for the experiment. The first baseline, ua-parser, refers to a security vulnerability involving the hijacking of a maintainers account to release malicious versions of an npm package. This incident highlighted a significant attack involving high-profile cases involving the hijacking of legitimate developer accounts, including ua-parser.[2] We selected ua-parser because it arose during a period when protestware incidents were becoming prominent. The second baseline, log4j, also known as CVE-2021-44228 or Log4Shell, represents a critical security vulnerability. It significantly impacted the software ecosystem, affecting over 17,000 packages (around 4% of the ecosystem) by December 19, 2021. Approximately 25% of these affected packages had fixed versions available.[3] Log4Shell was assigned a maximum severity score of 10 on the Common Vulnerability Scoring System (CVSS) scale.[4] We chose log4j due to its extensive impact and the rapid response from the open source community. To guide our study, we formulate the following two research questions:

– **RQ1: To what extent do developers engage in protestware discussions?**

[1] https://nvd.nist.gov/vuln/detail/cve-2022-23812

[2] https://t.ly/pdVPa

[3] https://security.googleblog.com/2021/12/understanding-impact-of-apache-log4j.html

[4] https://logging.apache.org/log4j/2.x/security.html

This question aims to quantify the spread of protestware discourse across the ecosystem and social media, determining both the speed and extent of discussions, which reflect community interest and concern.
Results show that developers comparatively engage in protestware discussions similar to security vulnerabilities. We find that it takes seven days to reach 136 developers and generate 245 posts for colors.js, and fourteen days to reach 155 developers and generate 720 posts for es5-ext. Overall, protestware discussions spread faster compared to vulnerability discussions during the early stages of the interruption on the GitHub platform, while being slower on social media platforms like Hackernews and Reddit.

- **RQ2: How can we characterize the contents of discussion regarding protestware between developers?** Qualitatively, we take a deeper investigation into the different narratives that revolve around the protestware, specifically focusing on the stances and communicative styles, themes emerging and mitigation strategies.
  - **RQ2a: What are the different stances and communicative styles that developers take when discussing protestware?** This question aims to measure how frequently developers express their stances on protestware and to identify the communicative styles they use in the discussions.
    Results include a classification of stances and communicative styles taken by developers. Among those who expressed a stance, "opposing" was more frequently associated with "direct" and "offensive/hate and toxic" communicative styles, while "favorable" stances were often expressed with "direct" styles.
  - **RQ2b: What different themes emerge from the discussions of protestware?** With this research question, we aim to categorize community reactions to protestware. The analysis helps us understand the content of developer posts, highlighting thematic elements that provide insights into the diverse perspectives within the community.
    Results include a taxonomy of themes that revolve around discussions of *rights and ethics*, *disseminate and response*, and *reputation*.
  - **RQ2c: What different kinds of mitigation strategies do developers provide for protestware?**
    This question aims to identify and analyze the suggestions provided by developers for mitigating the impact of protestware in software projects.
    Results show a classification of protestware mitigation. We found that 35% of the posts provided technical instructions for mitigating protestware, with key strategies including dependency management (11%), addressing license issues (7%), and suggesting alternatives (3%).
- **RQ3: To what extent do developers abandon protestware in their software?** We ask this question to investigate the direct influence of protestware on developers' decisions. We aim to assess whether protestware leads to the abandonment or continued usage of dependencies, which reflects the perceived severity or acceptance of the protest.
  Results show developers are less likely to abandon the protestware (i.e., out of 146 repositories that depend on colors.js, only 15% abandoned).
  Insights from the study indicate how protestware discussions are multifaceted and diverse. Potential tool support includes early detection, distilling technical mitigation instructions, and potential to moderate toxic-side effects of protestware discussions. The study's contribution is the first large-scale exploration into protestware, presenting a taxonomy of narratives that drive this emerging phenomenon.

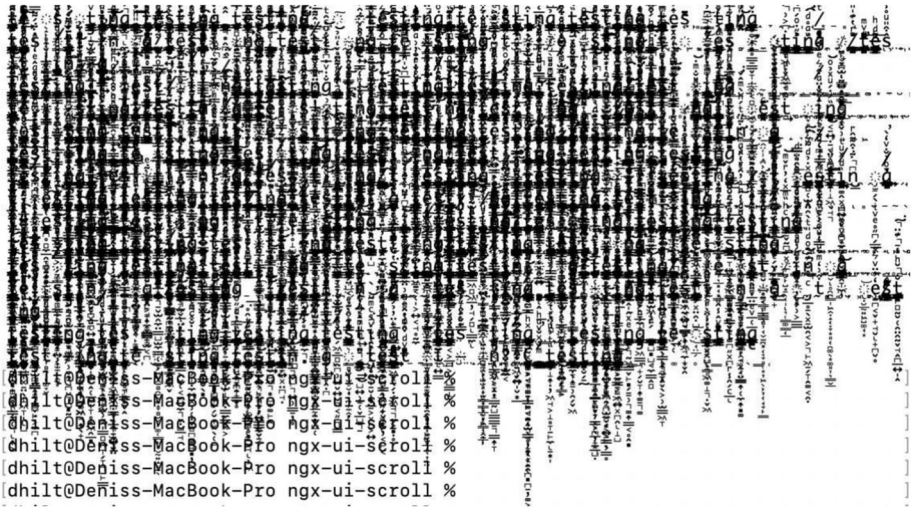

Fig. 1 Output of protestware colors.js

2 Protestware and Vulnerabilities

2.1 Case Studies

Specifically, we focus on two prominent instances of protestware (color.js and es5-ext). We selected these two cases due to their significant engagement within the community. colors.js, with over 3.3 billion downloads and dependency in more than 19,000 projects, has garnered substantial attention throughout its lifetime.[5] Similarly, es5-ex maintains a consistent weekly download count exceeding million, underscoring its ongoing relevance and widespread use.[6] Although the *node-ipc* and *Faker.js* libraries are among the more infamous cases of protestware, these projects have been closed or replaced, with most historical information removed from both GitHub and the broader Internet. To fairly compare the disruption, we in addition introduce two well-known security vulnerabilities (ua-parser and log4j) as the baseline.

colors.js - Protest Against Fortune 500 Companies Marak's motivations were explained on his personal blog from April 2021, entitled "Monetizing Open-source is problematic". He became well-known for his statement on the now-banned repository, Faker.js:

> *"No more free work from Marak*
> *- Pay Me or Fork This"*

The reaction from the open-source community has been diverse, sparking news articles and discussions on protestware. Marak's actions have stirred debates on many fronts, from the need for testing to the rights and responsibilities of code owners. As shown in Fig. 1, Marak also turned his colors.js library into protestware, leading to GitHub banning his action. A subsequent series of events opened a discussion on whether Marak's decision to self-sabotage his library was justified.

[5] https://blog.sonatype.com/npm-libraries-colors-and-faker-sabotaged-in-protest-by-their-maintainer-what-to-do-now

[6] https://checkmarx.com/blog/new-protestware-found-lurking-in-highly-popular-npm-package/

> *What? I'm totally on his side! Open-source maintainers should be supported by the companies who use their work. I hope they work out proper licensing deals, as suggested in the video.*

es5-ext - Protest against the Ukraine War Similarly, the popular NPM package es5-ext, with nearly 13 million weekly downloads, was updated to include a "Call for peace" message targeting machines configured to Russian time zones. Snippets of the code[7] are shown in Listing 1, which is only claimed to be invoked during testing. The maintainer's thoughts are captured in the following quote:

> *I'm opening this issue to serve as a single place to serve any (possibly constructive) discussions, towards the anti-Russian invasion on Ukraine manifest that's part of this package. As I mentioned in 28de285 whole point of this manifest, is to urge all Russians that see it, to seek reliable sources of information of what's going on, as you won't get it from official media sources in Russia. It's very important for you and the whole world that you realize what's really going on. Note that I'm affected by the situation directly, and it's why I've decided to put this manifest in...*

Listing 1 Protestware snippet to identify user location

```
// Broadcasts "Call for peace" message when package is installed in Russia, otherwise no-op
...
try {
    if (
        [
            "Asia/Anadyr", "Asia/Barnaul", "Asia/Chita", "Asia/Irkutsk", "Asia/Kamchatka",
            "Asia/Khandyga", "Asia/Krasnoyarsk", "Asia/Magadan", "Asia/Novokuznetsk",
            "Asia/Novosibirsk", "Asia/Omsk", "Asia/Sakhalin", "Asia/Srednekolymsk", "Asia/Tomsk",
            "Asia/Ust-Nera", "Asia/Vladivostok", "Asia/Yakutsk", "Asia/Yekaterinburg",
            "Europe/Astrakhan", "Europe/Kaliningrad", "Europe/Kirov", "Europe/Moscow",
            "Europe/Samara", "Europe/Saratov", "Europe/Simferopol", "Europe/Ulyanovsk",
            "Europe/Volgograd", "W-SU"
        ].indexOf(new Intl.DateTimeFormat().resolvedOptions().timeZone) === -1
....
```

3 Data Collection

In this section, we will describe the process of data preparation from both GitHub and the social media platform.

Protestware start and react points To capture all protestware discussions and their reactions, we first needed to identify the *start point* of each event. Hence, we looked at the actual commit where the protester inserted the protest code into the code base. As shown in Fig. 2, the code containing the protestware was committed on 2022.01.08 for colors.js and 2022.03.08 for es5-ext. For our baselines, we used the GitHub advisory's reported date as the start point for that vulnerability (2021.10.23 for ua-parser and 2021.12.10 for log4j).

[7] https://github.com/medikoo/es5-ext/blob/main/_postinstall.js

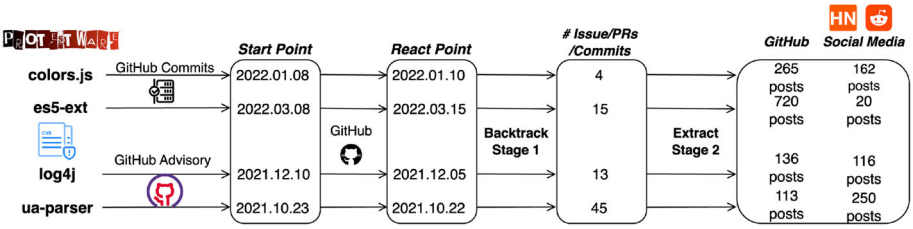

Fig. 2 An Overview of the Data Collection Process

In terms of the *reaction points*, we searched for the first issue that was raised in the protestware repository. As such, we identified the following issues as the reactions from GitHub for all four studied cases: (i.e., 2022.01.10 for colors.js[8], 2022.03.15 for es5-ext[9], 2021.12.05 for log4j[10], 2021.10.22 for ua-parser[11]).

Recursive Backtracking. To measure the spread of the protestware reactions throughout the rest of GitHub and social media, we implemented a recursive backtracking algorithm to extract any links (i.e., Issues, PRs, and commits). Note that in this paper, we refer to all conversations inside issues, PRs and commits as posts. We used the GitHub Rest API[12] to retrieve corresponding metadata including the post body. In compliance with GitHub's download rate limitations, we systematically downloaded the data over a period from 24th April to 30th June, 2023. Our backtracking followed two stages:

Backtrack Stage 1 - Recursive collection of all GitHub links. We applied a regular expression to capture all links, e.g., `https://github.com/([^/]+)/([^/]+)/issues/([\d]+)` for identifying any hyperlinks (similarily for PRs and commits) in the posts from the protester's repositories. As shown in Fig 2, we obtained four links for the Marak case (two issues and two commits across five repositories), three links for es5-ext case (two issues and one commit across two repositories), 13 links for log4j case (5 PRs and 8 commits across one repository), and 26 links for ua-parser case (14 issues, 8 PRs, and one commit across four repositories).

Extract Stage 2 - In this stage, we cross-reference mentions in all backtracked posts by collecting a series of events from GitHub links collected in Backtrack Stage 1 . GitHub provides this information, identifying any issues, pull requests (PRs), and commits that were referenced or cross-referenced (GitHub Reference 2023; GitHub Cross-Reference 2023) from another issue, pull request, or commit within the timeline. All collected posts are depicted in Fig 2.

Discussions in Social Media To measure the spread in other outlets such as social media, we chose to focus on Reddit and Hacker News, since these two media platforms are popular for developers to submit posts related to software development and they have been commonly studied in the context of software engineering research (Aniche et al. 2018). To remove noise from non-developers and ensure data quality, we input the issues, PRs, and commits

8 https://github.com/Marak/colors.js/issues/289

9 https://github.com/medikoo/es5-ext/issues/116

10 https://github.com/apache/logging-log4j2/pull/608

11 https://github.com/faisalman/ua-parser-js/issues/536

12 https://docs.github.com/en/rest?apiVersion=2022-11-28

referenced as the start point for the protestware. Note that in our study, the comment and its reply are regarded as two distinct posts. We retrieved three threads from Reddit and one thread from Hacker News. To collect their posts, we utilized various APIs, such as the Python Reddit API Wrapper for Reddit,[13] and the Request library for Hacker News. As shown in Fig. 2, we ended up with 540 posts.

4 Empirical Results

We now present the approaches and the results of our proposed research questions.

4.1 Impact of Disruption (RQ1)

To address RQ1, we answer the question as shown below.

Spread throughout the Ecosystem (RQ1)

Inspired by related work (Kanuri et al. 2018), we defined social media metrics to measure the engagement and how quickly discussions of protestware spread within the GitHub ecosystem and the external social media. We use the four origin issues as the starting point of each studied case and use the following definitions:

- *Post.* All conversations are regarded as posts for any issue, PR, commit, and social media posts. Each post has a timestamp, the post context in the text, and the developer who is the author of the post.
- *Engaged developer:* This is a count of unique developers who have authored any protestware posts after the start point. Hence, we only count the first instance of the engaged developer post.
- *Engaged post:* This is the count of unique posts from the start point of the protestware. Similar to "engaged developers", this includes only posts created after the start point, removing duplicates.

Based on the above definitions, we now introduce our engagement metrics.

- *Developer engagement metric.* This measures the number of developers engaged within one, seven, and fourteen days from the start point of the disruption.
- *Post engagement metric.* This measures the number of posts generated within one, seven, and fourteen days from the start point of the disruption.
- *Social media post engagement metric (Hackernews and Reddit).* This measures the number of posts made on social media platforms (Hackernews and Reddit) within one, seven, and fourteen days from the start point of the disruption.

Discussions of protestware spread like wildfire on GitHub, but not on social media.

Table 1 summarizes the engagement metrics. Our analysis shows that while protestware discussions spread quickly on the GitHub platform, security vulnerabilities tend to spread faster on social media. For example, on GitHub, colors.js engaged 136 developers within seven days, whereas ua-parser engaged only 50 developers at the same time. However, ua-parser generated 146 posts in just one day, illustrating the faster dissemination of vulnerability discussions compared to protestware on social media platforms.

[13] https://github.com/praw-dev/praw

Table 1 Summary statistics for time-to-engage (RQ1)

	colors.js	es5-ext	log4j	ua-parser
Developer Engagement Metric				
- one day	9 devs	-	24 devs	21 devs
- seven days	136 devs	37 devs	55 devs	50 devs
- fourteen days	139 devs	155 devs	56 devs	56 devs
Post Engagement Metric				
- one day	11 posts	-	32 posts	24 posts
- seven days	245 posts	148 posts	76 posts	70 posts
- fourteen days	250 posts	720 posts	77 posts	77 posts
Hackernews Post Engagement Metric				
- one day	7 posts	-	19 posts	7 posts
- seven days	12 posts	-	41 posts	7 posts
- fourteen days	12 posts	-	41 posts	7 posts
Reddit Post Engagement Metric				
- one day	53 posts	-	2 posts	146 posts
- seven days	129 posts	-	50 posts	191 posts
- fourteen days	130 posts	1 post	51 posts	191 posts

Note: devs = developers

RQ1 Summary

Our quantitative results indicate that discussions about protestware (i.e., colors.js and es5-ext) tend to spread more quickly on the GitHub platform, while security vulnerabilities (i.e., ua-parser and log4j) engage faster on social media. For example, within the first seven days, colors.js engaged 136 developers and generated 245 posts, demonstrating its significant impact on the GitHub ecosystem. In contrast, security vulnerabilities like ua-parser spread faster on social media, with 146 posts engaged in just one day.

4.2 Characteristics of Posts (RQ2)

To address RQ2, we answer three sub-research questions by classifying the topics of discussion in the conversations about protestware. We conducted an in-depth assessment of developers' positions and the nature of their posts. Due to the complex and nuanced nature of protestware descriptions in natural language (Cheong et al. 2023; Kula and Treude 2022), it is challenging to use any existing tools for detection. Hence, we opted for a manual coding approach. Note that for RQ2, since we focus only on protestware posts, we exclude the baseline cases (i.e., ua-parser and log4j) from our analysis. Additionally, to reduce noise in the dataset, we filtered the data by including only posts that contain links directly related to colors.js and es5-ext.

Qualitative Dataset

Table 2 shows that our dataset contains a total of 1,640 posts (1,458 from GitHub and 182 from social media). Following prior works (Hata et al. 2019; Wang et al. 2021), we adopted a systematic manual coding approach with multiple rounds to identify the relevant posts.

Table 2 Dataset to answer the Qualitative Analysis in RQ2

	colors.js	es5-ext
From GitHub		
- # protestware repo posts (1,458)	797	661
From Social Media		
- # posts from RQ1 (182)	162	20
After Filtering		
- Sum # posts for RQ2 (GitHub + Social Media)	578	106

Since the quantitative analysis in RQ1 is an overestimation, especially with backtracking the mentions in our approach, we focus only on the results of the recursive backtracking from stage 1.

Emerging themes Since manual analysis of the posts is a tedious approach, we adopted a systematic but iterative process. For all sub-research questions, we then took a holistic approach to conduct the classifications with the three different dimensions in mind (i.e., stances and communicative styles, themes, and mitigation strategies). Our iterative process is broken into three processes. Note that since combining all sub-questions, each classification was derived independently.

- *Initial Coding*. The goal of the initial coding was to ensure we had saturation and coverage of all themes. Initially, two researchers reviewed 20 posts together and discussed potential categories and codes, leading to the development of a preliminary code schema. Based on this schema, three researchers collaboratively coded 30 posts, achieving Kappa values of 0.73 for technical information, 0.80 for dissemination and response, 0.75 for stance, 0.67 for reputation, 0.67 for communicative style, and 0.78 for rights and ethics. These agreements were considered moderate (Viera and Garrett 2005). The researchers then distributed the rest of the dataset among themselves.
- *Round-table discussions for consensus*. After completing the individual coding, the researchers reconvened to discuss and resolve all highlighted uncertainties, resulting in a total of 139 posts being reviewed. The three researchers sat together to discuss and reach a consensus on all codes, especially in cases where the coders were not confident. After extensive discussion, the three researchers decided on each classification.
- *Final consensus and definitions refinement*. After all classifications were made, the three researchers then carefully refined the definitions of each code to ensure that the themes were properly represented.

Developers were primarily opposing the protestware, using either direct or offensive/hate and toxic communicative styles (RQ2a) The first dimension of protestware discussions is classified by both stance and communicative style. As shown in Fig. 3, where we applied co-occurrence analysis, most developers in the discussions did not express a clear stance (68%). Among the 32% who did, the majority were opposed to protestware (21%), with their communication often being direct, offensive, or characterized by hate and toxicity (9%). This indicates a polarized and frequently harsh discussion environment surrounding protestware issues. To assess the significance of the relationship between communicative styles and stances, we applied Pearson's Chi-Squared test (Pearson 1900), which is commonly used to evaluate relationships between categorical variables (Wang et al. 2023). The test confirmed a significant relationship between communicative styles and stances, with a p-value of < 0.001, indicating that the association between communicative styles and stances

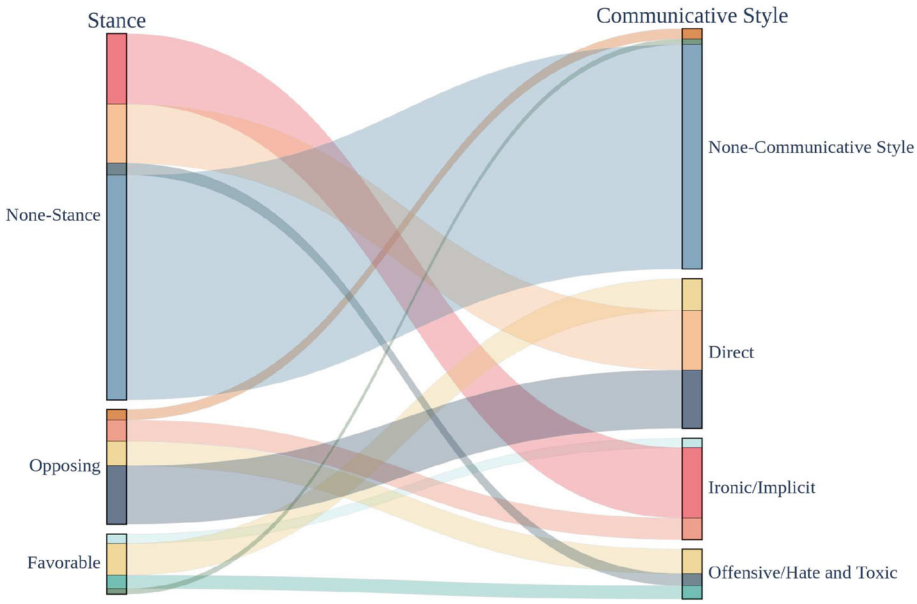

Fig. 3 Distribution between the stance and communicative style

is highly significant. We now provide the definitions and examples for each of our codes as follows:

- **Stance**
  - *Favorable (11%).* The post expresses support for the protester and the actions of the protestware, for example "Creative choices like this (while disruptive) do keep us alert. I'm sympathetic to all of us who this impacted, but I'm in support of playfulness, developer wellbeing and personal agency for contributors and maintainers such as @Marak".
  - *Opposing (21%).* The post expresses opposition to the protester and the actions of the protestware, for example "There are 44 contributors to this project, it is their code, not his."
  - *None (68%).* The post did not express support or opposition to the protestware, or it is hard to tell, for example "Ah isn't that related to fakerjs? Wasn't it maintained by the same person or?"
- **Communicative Style**
  - *Offensive/Hate and Toxic (9%).* The post contains insults or profanity, or the post is characterized by malicious intent, for example "Wow, another NPM shitshow. The major problem is why the other libraries don't fix their dependency versions."
  - *Ironic/Implicit (19%).* The post contains expressions that have opposite literal meanings to real meanings. The implicit nature refers to the post containing anecdotal narratives, for example "v1.4.44-liberty-2 is working great for me, @Marak. Thanks for pushing this update quickly, my console has never looked better."
  - *Direct (28%).* The post contains expressions that take a firm and conclusive stance on an issue, clearly stating an opinion with no alternative meanings or interpretations, for

example "He published malicious code when he has > 20 million weekly downloads, not acceptable."
- *None (44%).* The post does not express any sentiments in terms of offensive, hate and toxic, ironic, implicit, explicit or direct, for example "Folks interested in what's going on with colors.js should have a look at the update from DABH (who had made the more recent and substantial changes) on GitHub. https://t.co/uMlIAjqiko"

Topics of Discussion (RQ2b) The second dimension, as shown in Fig 4 is a taxonomy of the topics discussed in relation to protestware. As shown in Fig 5, the topic highlights the different focuses and concerns developers have when discussing protestware, particularly in terms of reputation and rights and ethics, the detailed categories are as follows:

- **Rights and Ethics**
  - *Legal (8%).* The post owner expresses the legal implications of the protestware, for example "He's using a MIT license, which grants the right for people to use his work commercially, for free."
  - *Moral (19%).* The post owner expresses ethical considerations and consequences of the protestware, for example "There are 44 contributors to this project, it is their code, not his."
  - *OSS Philosophy (15%).* The post expresses views that are related to the ethical philosophy behind open source software, for example "Also known as "contribute back to the community or pay up" :laughing: "
  - *None (58%).* The post does not express any rights or ethical considerations regarding the protestware, for example "github and npm are two different things - you can publish to npm without ever pushing the code to github"

- **Disseminate and Response**
  - *True (21%).* The post contains external links of information related to the protestware. It does not contain any views of the post owner, for example "it's no bug, they vandalized their own project: https://github.com/Marak/colors.js/commit/074a0f8ed0c31c35d13d28632bd8a049ff136fb6"

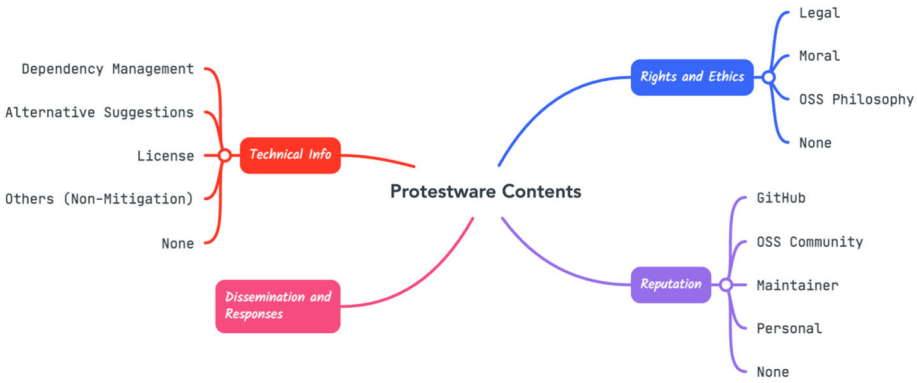

Fig. 4 A mind map of the themes emerging from protestware discussions

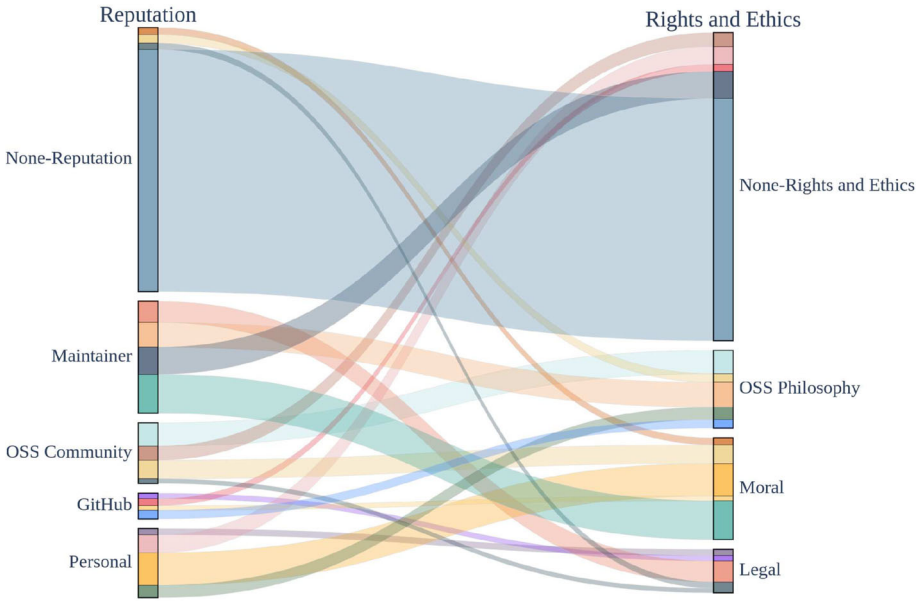

Fig. 5 Distribution of the different reputations and their rights and ethics expressed

  - *False (79%).* The post does not contain any external links that are information on the protestware, for example "Ah isn't that related to fakerjs? Wasn't it maintained by the same person or?"

- **Reputation**
  - *Maintainer (21%).* The post is directed to the protester's reputation as a maintainer in the OSS community, for example "That can happen too: Maintainer of a popular #javascript library colors.js intentionally pushed an infinite loop into the code, crashing thousands of other libraries that depends on it. here's the malicious commit: https://github.com/Marak/colors.js/commit/074a0f8ed0c31c35d13d28632bd8a049ff136fb6 #javascriptdeveloper"
  - *OSS Community (11%).* The post is directed to the OSS community reputation that is affected by the protestware, for example "Wow, another NPM shitshow. The major problem is why the other libraries don't fix their dependency versions."
  - *Personal (13%).* The post is a personal attack on the protester reputation. Not on their role as a maintainer, for example "Also.. just a note.. colors.js actually had co-maintainers. He removed their access so they couldn't fix things. That's an extra level of cringe for me."
  - *GitHub (5%).* The post is directed to the reputation of the GitHub Platform, for example "Github's responsibility should have been to treat him like they treat everyone else and when they removed him from the platform they removed his content and code."
  - *None (50%).* The post is not related to reputation, for example "Color contains malware as far as I know? Doesn't it?"

Developers provide technical information that sometimes provides mitigation instructions (RQ2c) The third dimension involves the classification of mitigation strate-

gies. As shown in Table 3, we classified technical posts into two categories: those that provide mitigation instructions and those that do not. Developers are more likely to provide mitigation strategies for colors.js compared to es5-ext. After discussion, we then were able to code the technical posts as the following:

- *With Technical Instructions*
  - *Alternative Suggestions (3%).* The post contains an alternative fix to the protestware, for example "For developers looking to migrate, alternatives of colors are: chalk, kleur, cli-color, colorette."
  - *License (7%).* The post contains some discussion on the licensing to mitigate the protestware, for example " He's using a MIT license, which grants the right for people to use his work commercially, for free."
  - *Dependency Management (11%).* The post suggests changing (upgrading/downgrading) dependencies to mitigate the protestware, for example "v1.4.1 is also broken v1.4.0 is working."
  - *Other (13%).* The post mentioned other methods to mitigate protestware that are not mentioned above, for example "I can't launch because of this issue. Assuming this never gets fixed, what's the proper way to force npm to use an older version of this dependency for 'http-server'? Edit the lock file?"
- *Without Technical Instructions*
  - *None (65%).* The post does not contain any technical suggestions, for example "Ah isn't that related to fakerjs? Wasn't it maintained by the same person or?"

RQ2 Summary

Our analysis reveals the diverse stances, communicative styles, discussion topics, and mitigation strategies that developers use when discussing protestware.

- Answering **RQ2a**, most developers (68%) did not express a clear stance, but those who did were primarily opposed to protestware (21%), using direct (28%) or offensive/hate and toxic (9%) communicative styles.
- Answering **RQ2b**, developers' discussions on protestware centered around the topics of moral (19%), OSS philosophy (15%), legal (8%), dissemination and response (21%), and reputation about maintainer (21%).
- Answering **RQ2c**, developers provided various mitigation strategies for protestware, including dependency management (11%), license discussions (7%), alternative suggestions (3%), and other technical instructions (13%), although 65% of posts lacked technical suggestions.

Table 3 Distribution of posts with Technical Information from RQ2c

	colors.js	es5-ext
With Technical Info		
- Other	84 (12%)	8 (1%)
- Dependency Management	77 (11%)	-
- License	46 (7%)	1 (0%)
- Alternative Suggestions	23 (3%)	-
Sum	**230** (34%)	**9** (1%)

4.3 Abandoning the Protestware (RQ3)

To understand the influence of protestware, we conducted a quantitative analysis to investigate whether developers who discussed protestware ended up abandoning the associated dependency. We analyzed all the repositories from our collected dataset, focusing on those that (i) are valid JavaScript repositories, and (ii) use protestware as a dependency. To achieve this, we adopted an approach similar to prior works (Cogo et al. 2019; Decan et al. 2017; Wittern et al. 2016), identifying projects that contain the `package.json` file. Within these JSON files, we then determined whether or not protestware was documented as a dependency. Our analysis revealed that 4,961, 1, and 5,449 repositories met the above criteria for colors.js, es5-ext, and ua-parser. As shown in the Table 4 all discussions for es5-ext come from a single repository. This is due to the stage one backtracking only identifying one repository. We confirmed that this was the es5-ext repository itself. To detect whether or not a repository abandoned the protestware, we use the PyDriller package[14] to mine the historical dependency changes, from the date of the disruption until the latest commit in our dataset. Inspired by the work of Wattanakriengkrai et al. (2022), we focus on the following four types of dependency changes: dependency add, remove, upgrade, and downgrade. In addition, we measure the popularity of the influenced repositories. Specifically, we employ the GitHub API to retrieve common popularity metrics, i.e., #Forks, #Commits, #Stars, and #Contributions (Borges et al. 2016).

Since Log4j is not written in JavaScript, we employed a separate approach. We used the backtrack stage 1 method to identify all related GitHub projects. We then filtered for projects primarily using Java and retrieved their dependency lists via the GitHub GraphQL API. This allowed us to check if these projects still use Log4j and determine the version in use.

Developers show a lower likelihood of abandoning protestware. Table 4 offers insights into the characteristics of repositories influenced by protestware and the distribution of dependency changes. We observe that the number of repositories dependent on the protestware, colors.js, is greater than the vulnerability baseline, ua-parser. For example, the number of dependent repositories for colors.js is 146, while the number for ua-parser is 18. Additionally, compared to the ua-parser, we find that the repositories affected by the protestware are relatively popular. The median values for the number of commits, stars, and contributors are 604, 124, and 23, respectively.

Table 4 also reveals that on one hand, the majority of influenced developers do not abandon the protestware dependency. In colors.js, only 15% of repositories abandoned the protestware, which is similar to the ua-parser and log4j where only 6% of repositories abandoned the vulnerable package. There are several causes for why they were not abandoned. Different to a vulnerability, protestware has two key qualities:

- The protester will not release a new version. Alternative mitigation include a new or forked library.
- The protestware is deemed benign and does not affect the library user.

There also maybe other reasons that are unknown to the authors. For example, in the case of the es5-ext, the manifest does not impact the behaviour of the software. However, in the case of colors.js, the reasons could be either the software is inactive or there is no alternative for mitigation, leaving the software hopeless to the remedy. That being said, these are all points of discussion and we can only report the current situation. Probably this might be the reasoning why developers are less likely to make changes to the protestware package. For ua-parser,

[14] https://github.com/ishepard/pydriller

Table 4 Influenced repositories that abandoned the protestware (RQ3)

	colors.js	es5-ext	ua-parser	log4j
Collected Dataset				
- # All collected repositories	17,457	1	17,280	20,900
Backtrack Stage 1 Repositories				
- # Same language repositories	4,961	1	5,449	512
- # Not dependent	4,815	0	5,431	483
- **# Dependent Repos**	**146**	**1**	**18**	**29**
Repo Characteristics				
- # Forks (Median-Max)	Med=46, Max=14,773	915	Med=142, Max=3,045	Med=25, Max=15,176
- # Commits (Median-Max)	Med=604, Max=35,552	9,670	Med=1,974, Max=35,627	Med=442, Max=11,495
- # Stars (Median-Max)	Med=124, Max=108,008	2,170	Med=445,Max=25,489	Med=46, Max=32,990
- # Contributors (Median-Max)	Med=23, Max=429	358	Med=22,Max=361	Med=11, Max=30
Detected Dependency Changes				
- No change (%)	77%	100%	6%	3%
- ✚ Added library (%)	0.5%	-	6%	-
- ✖ Removed library (%)	15%	-	6%	6%
- ⌃ Upgraded library (%)	7%	-	82%	54%
- ⌄ Downgraded library (%)	0.5%	-	-	-

82% of the repositories and log4j, 54% of the repositories addressed the vulnerability by upgrading to a newer version of the library.

RQ3 Summary

Our results indicate that the affected repositories were relatively less likely to abandon the protestware when compared to vulnerabilities. For example, out of the 146 repositories that depend on colors.js, 15% have abandoned the dependency, which is higher than the 6% abandonment rate for ua-parser and log4j. This suggests that although there is some abandonment, it is not as significant as for vulnerabilities, possibly due to the lack of a defined mitigation strategy for protestware or because some protestware is deemed benign.

5 Lessons Learned

Based on our results, we now discuss the implications and challenges for both developers and researchers.

R1: Protestware discussions are multi-faceted and serve a diverse community of related entities.

As demonstrated in our thematic analysis in RQ2b, it is evident that arguments from both sides are valid within their respective contexts (e.g., OSS philosophy and legal issues and rights). A solution would require a more comprehensive understanding of the bigger picture. Therefore, we urge the open-source community and all related stakeholders, including GitHub and open-source developers, to consider all viewpoints. These considerations should also be acknowledged by potential protesters and dogmatic developers.

Our taxonomy could be utilized to understand the possible narratives that the protestware could navigate and how they could be resolved. For example, most of the posts discuss rights and ethics, which could be discussed in a more open-minded way by the open-source community, especially understanding opposing thoughts. One example is researchers and practitioners need to determine the most effective communication channels for developers to express their views to their OSS community.

R2: Benign protestware is not abandoned. As demonstrated in RQ1, protestware is less likely to be abandoned. For example, in colors.js, only 15% of repositories abandoned the protestware. Our results also indicate that protestware stimulates diverse discussions. These findings suggest that, when properly managed, protestware could be a viable approach to raising awareness of social needs, with developers becoming more receptive to individuals who want to highlight such issues. However, it is still too early to draw definitive conclusions.

In our case study, we find that the protester created a manifest file to document his protest. One idea is for GitHub to create a channel similar to the security.md policies file[15], so that protesters can peacefully protest in benign manner.

R3: Potential for Early Protestware Detection. As shown in RQ1 (Table 1), we found that communication outlets and media platforms, such as Reddit, were able to quickly detect the protestware. Studies have demonstrated the effectiveness of NLP and sentiment analysis in detecting and analyzing social media content Pak et al. (2010); Sarker et al. (2020). One implication for researchers and practitioners is the need for tool support to monitor

[15] https://docs.github.com/en/code-security/getting-started/adding-a-security-policy-to-your-repository

and promptly detect such disruptions before they spread throughout the ecosystem. Early detection can help users to prepare for the disruption, also the OSS community can prepare to handle the different conversations and narratives that might come into play during the protestware.

R4: Protestware discussions contains technical mitigation instructions. Results from RQ2c indicate that many of the discussions likely include instructions (e.g., dependency management and alternative suggestions) on how to mitigate and resolve issues influenced by protestware. An actionable implication could be that tool support is proposed to assist developers in migrating away from the protestware by utilizing the crucial information distilled from the discussions. Similar to the vulnerability advisory, there could be a similar kind of board that enables developers to find mitigation strategies. Especially, using our taxonomy, such automatic tools similar to the Dependabot could be used to assist developers in quickly updating their dependencies or licenses.

R5: Toxic conditions are a side-effect of protestware. Manual analysis in RQ2a revealed that many of the opposing stances expressed by developers were toxic in nature. Several studies indicate that toxic interactions and uncivil language often lead to the failure of many new and longstanding OSS projects (Miller et al. 2022). Thus, to promote inclusivity and diversity in software development, we suggest implementing mechanisms or policies to prevent protestware from negatively impacting the OSS community. Recently, there has been a rise in tools like GitHub action bots used to automatically detect toxic comments in different OSS communities. Including the corpus of our protestware discussions would enhance the dataset used to train automatic toxicity tools.

R6: Enhancing Security and Trust through Awareness. As discussed in RQ2b, our study underscores the importance of being aware of protestware and its implications. Specifically, 21% of posts disseminate external links related to protestware, emphasizing the community's role in spreading awareness and information. Therefore, it is crucial to understand the implications of protestware. Practitioners can develop security protocols and training programs to better prepare their teams for potential protestware incidents, thereby enhancing overall software security and trust.

R7: Promoting Sustainable Software Development Practices. As highlighted in RQ2c, our study emphasizes the need for sustainable software development practices that can withstand influences like protestware. The results in Table 4 show that a significant percentage of developers do not abandon protestware, with only 15% of repositories abandoning colors.js, suggesting the necessity for sustainable practices to handle such scenarios, considering that some of these repositories may be inactive. Practitioners should focus on creating robust and flexible development environments that can adapt to changes and challenges. By implementing sustainable practices, organizations can ensure the long-term health and stability of their software projects, even in the face of unexpected disruptions.

6 Threats to Validity

One of *external threat* to the validity of our findings is the ability to generalize them, as our work is presented as a multiple case study, encompassing colors.js and es5-ext. We acknowledge that observations may not be generalized to other protestware cases and ecosystems. As the first study to systematically analyze the discussions of protestware, we argue that these two cases can be pioneering baselines that represent society issues of economic and world

politics. We envision more studies to follow our work, exploring how to reproduce our results on other protestware in the future.

In terms of *construct threats*, a major threat arises on how we systematically retrieve posts from both GitHub and social media. We understand that a different methodology, such as different search methods may change the amount of data collected. Furthermore, we only consider two platforms: Reddit and Hacker News, and it is possible that we may have an underestimation of the true impact. Although twitter (X) is more prevalent, due to recent events, we were unable to utilize X new API to collect information.[16] That being said, we selected a method that would return fewer false positives. Hence, as future work, we plan to expand our work to include a more robust and dynamic method to collect our dataset. Additionally, a *construct threat* in our study arises from the potential subjectivity and bias introduced during the coding process. Despite our collaborative efforts to develop and refine the code schema, the inherent subjectivity in qualitative coding means that independent coders might find it challenging to replicate our taxonomy and achieve consistent results. This poses a limitation to the replicability of our findings.

We identify two key *internal threats*. The first threat exists in the selection of the metrics to measure the disruption of protestware in RQ1. Although the engagement metric quantifies speed, we cannot make any statistical claims. The second threat occurs during our multiple manual analyses of posts in RQ2, where the codes or themes may be mislabeled due to their subjective nature. While we acknowledge that qualitative measures are prone to subjective bias, we implemented a typical systematic method established in prior works and involved more than one coder in each manual analysis.

7 Related Work

We now position our work with respect to the related literature.

Software Development Reliance on Software Ecosystems Modern software development heavily relies on third-party libraries, which is evident by supply chain attacks (Vu et al. 2020; Zahan et al. 2022; Zimmermann et al. 2019). A recent report from SSonatype. (2021) stated that supply chain attacks have dramatically increased 650% in 2021 on top of year-over-year growth of 430% in 2020. Ohm et al. (2020a) studied the three popular package ecosystems (npm, PyPI, and RubyGems) and presented a taxonomy of attack vectors and a dataset of malicious software packages. Hou and Jansen (2023) demonstrated that stakeholders' trust is frequently violated by bad actors and crippling vulnerabilities in the software supply chain. At the same time, several automated solutions have been proposed to protect against supply chain attacks (Ohm et al. 2020b; Scalco et al. 2022). There have also been several tools proposed (Gonzalez et al. 2021; Sejfia and Schäfer 2022) for automatically detecting malicious npm packages and their evaluation results show promise in practice.

Economic Incentives in Open Source Bounties are frequently used to attract developers and motivate them to complete diverse software development tasks, such as software vulnerabilities (Finifter et al. 2013) and bug reports (Hata et al. 2017). Krishnamurthy and Tripathi (2006) provided an overview of bounty programs in FLOSS and suggested that bounty hunters' responses are related to the workload. Finifter et al. (2013) analyzed vul-

[16] https://developer.twitter.com/en/docs/twitter-api/migrate/whats-new

nerability rewards programs for Chrome and Firefox and reported that such programs are economically effective, compared to the cost of hiring full-time security researchers. Maillart et al. (2017) recommended that project managers should dynamically adjust the value of rewards according to the market situation. Zhou et al. (2020b) studied the association between bounties and the issue-addressing likelihood of GitHub issue reports. In the context of the question-answering process, Zhou et al. (2020a) observed that questions are likely to attract more traffic after receiving a bounty. The latest works (Shimada et al. 2022; Zhou et al. 2022) also investigate GitHub Sponsors, a service allowing developers to accept donations. Specifically, Shimada et al. (2022) found that sponsored developers are more active than non-sponsored developers.

Social Good in Software Engineering Based on the positive impact of software on society in recent years, Ferrario et al. (2014) defined SE for "social good" as the development, maintenance, and sustainability of software aimed at promoting social change. Along with this concept, Huang et al. (2021) specifically targeted Open Source Software for Social Good (OSS4SG), which refers to OSS projects that benefit society. In April 2020, GitHub's Social Impact Sector released a report indicating a tremendous increase in demand for open-source projects with social impact, and a lack of research and resources related to them. Dekhtyar et al. (2020) recommend that SE conferences should focus on activities that benefit society and that these activities should be an integral, valued, and recognized part of the conference programs. Protestware provides a different lens by which we can view different relationships of OSS to such societal issues.

8 Conclusion and Future Work

At this early stage, it is unsure whether or not protestware is here to stay in the open source software ecosystem. With the heavy reliance on the ecosystem and its supply chain, the mere presence of protestware is a reminder that software development is becoming more and more intertwined with society and its issues, ranging from economic to political. In this first large-scale exploration into protestware, we present the different narratives that drive this emerging phenomenon. The results demonstrate that developers are less likely to abandon the protestware. Developers are often vocal about their opinions on protestware and opinions are multifaceted with different contexts, perceptions, and tones. Additionally, they frequently provide technical information that can include mitigation instructions.

This work lays the foundation for further research and provides various opportunities. These future research opportunities encompass not only providing developers with insights into the varying perspectives but also offering actionable guidance to foster constructive dialogue and reduce toxicity in these sensitive discussions. Such frameworks could transform protestware from a potential point of contention into an instrument for positive social change, aligning the open-source community with broader societal values and goals.

Acknowledgements This work was supported by JST SICORP Grant Number JPMJSC2206, JST BOOST Grant Number JPMJBS2423, JSPS KAKENHI Grant Numbers JP20H05706, JP23K16864, JP23K22779, and JST PRESTO Grant Number JPMJPR22P6.

Data Availability The datasets generated during and/or analysed during the current study are available at https://doi.org/10.5281/zenodo.8207976

Declarations

Conflict of Interest The authors declare that Raula Gaikovina Kula, Hideaki Hata, and Christoph Treude are members of the EMSE Editorial Board. All co-authors have seen and agree with the contents of the manuscript and there is no financial interest to report.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

Aniche M, Treude C, Steinmacher I, Wiese I, Pinto G, Storey MA, Gerosa MA (2018) How modern news aggregators help development communities shape and share knowledge. In: Proceedings of the 40th international conference on software engineering, pp 499–510

Bellovin SM (2022) Open source and trust. IEEE Secur Priv 20(02):107–108

Borges H, Hora A, Valente MT (2016) Understanding the factors that impact the popularity of github repositories. In: 2016 IEEE international conference on software maintenance and evolution (ICSME), IEEE, pp 334–344

Cheong M, Kula RG, Treude C (2023) Ethical considerations towards protestware. arXiv:2306.10019

Chinthanet B, Reid B, Treude C, Wagner M, Kula RG, Ishio T, Matsumoto K (2021) What makes a good node. JS package? investigating users, contributors, and runnability. arXiv:2106.12239

Cogo FR, Oliva GA, Hassan AE (2019) An empirical study of dependency downgrades in the npm ecosystem. IEEE Trans Softw Eng 1–1

Decan A, Mens T, Claes M (2017) An empirical comparison of dependency issues in OSS packaging ecosystems. In: 2017 IEEE 24th International conference on software analysis, evolution and reengineering (SANER), pp 2–12

Ferrario MA, Simm W, Newman P, Forshaw S, Whittle J (2014) Software engineering for 'social good': integrating action research, participatory design, and agile development. In: Companion Proceedings of the 36th international conference on software engineering, Association for Computing Machinery, New York, NY, USA, ICSE Companion 2014, p 520–523, https://doi.org/10.1145/2591062.2591121,

Ferrario MA, Simm W, Newman P, Forshaw S, Whittle J (2014) Software engineering for 'social good': integrating action research, participatory design, and agile development. In: Companion Proceedings of the 36th international conference on software engineering, Association for Computing Machinery, New York, NY, USA, ICSE Companion 2014, pp 520–523. https://doi.org/10.1145/2591062.2591121

Finifter M, Akhawe D, Wagner D (2013) An empirical study of vulnerability rewards programs. In: 22nd USENIX security symposium (USENIX Security 13), pp 273–288

Ghofrani J, Heravi P, Babaei KA, Soorati MD (2022) Trust challenges in reusing open source software: an interview-based initial study. In: Proceedings of the 26th ACM international systems and software product line conference-volume B, pp 110–116

GitHub Cross-Reference (2023). https://docs.github.com/en/webhooks-and-events/events/issue-event-types#cross-referenced

GitHub Reference (2023). https://docs.github.com/en/webhooks-and-events/events/issue-event-types#referenced

Gonzalez D, Zimmermann T, Godefroid P, Schäfer M (2021) Anomalicious: automated detection of anomalous and potentially malicious commits on github. In: 2021 IEEE/ACM 43rd international conference on software engineering: software engineering in practice (ICSE-SEIP), IEEE, pp 258–267

Hata H, Guo M, Babar MA (2017) Understanding the heterogeneity of contributors in bug bounty programs. In: 2017 ACM/IEEE international symposium on empirical software engineering and measurement (ESEM), IEEE, pp 223–228

Hata H, Treude C, Kula RG, Ishio T (2019) 9.6 Million links in source code comments: purpose, evolution, and decay. In: 2019 IEEE/ACM 41st international conference on software engineering (ICSE), IEEE, pp 1211–1221

He H, Xu Y, Ma Y, Xu Y, Liang G, Zhou M (2021) A multi-metric ranking approach for library migration recommendations. 2021 IEEE International Conference on Software Analysis. Evolution and Reengineering (SANER), IEEE, pp 72–83

Hou F, Jansen S (2023) A systematic literature review on trust in the software ecosystem. Empir Softw Eng 28(1):8

Huang Y, Ford D, Zimmermann T (2021) Leaving my fingerprints: motivations and challenges of contributing to oss for social good. In: 2021 IEEE/ACM 43rd international conference on software engineering (ICSE), pp 1020–1032. https://doi.org/10.1109/ICSE43902.2021.00096

Kanuri VK, Chen Y, Sridhar S (2018) Scheduling content on social media: theory, evidence, and application. J Market 82(6):89–108

Krishnamurthy S, Tripathi AK (2006) Bounty programs in free/libre/open source software. In: The economics of open source software development, Elsevier, pp 165–183

Kula RG, Treude C (2022) In war and peace: the impact of world politics on software ecosystems. In: Proceedings of the 30th ACM joint European software engineering conference and symposium on the foundations of software engineering, pp 1600–1604

Kula RG, German DM, Ouni A, Ishio T, Inoue K (2018) Do developers update their library dependencies? an empirical study on the impact of security advisories on library migration. Empir Softw Eng 23:384–417

Maillart T, Zhao M, Grossklags J, Chuang J (2017) Given enough eyeballs, all bugs are shallow? revisiting eric raymond with bug bounty programs. J Cybersecur 3(2):81–90

Massacci F, Sabetta A, Mirkovic J, Murray T, Okhravi H, Mannan M, Rocha A, Bodden E, Geer DE (2022) "Free" as in freedom to protest? IEEE Secur Priv 20(5):16–21

Miller C, Cohen S, Klug D, Vasilescu B, KaUstner C (2022) " Did you miss my comment or what?" understanding toxicity in open source discussions. In: Proceedings of the 44th international conference on software engineering, pp 710–722

Ohm M, Plate H, Sykosch A, Meier M (2020a) Backstabber's knife collection: a review of open source software supply chain attacks. In: Detection of intrusions and malware, and vulnerability assessment: 17th international conference, DIMVA 2020, Lisbon, Portugal, June 24–26, 2020, Proceedings 17, Springer, pp 23–43

Ohm M, Sykosch A, Meier M (2020b) Towards detection of software supply chain attacks by forensic artifacts. In: Proceedings of the 15th international conference on availability, reliability and security, pp 1–6

Pak A, Paroubek P et al (2010) Twitter as a corpus for sentiment analysis and opinion mining. LREc 10:1320–1326

Pearson K (1900) X. on the criterion that a given system of deviations from the probable in the case of a correlated system of variables is such that it can be reasonably supposed to have arisen from random sampling. The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science 50(302):157–175

Sachs J, the Faker Team (2022) An update from the Faker team | Faker — fakerjs.dev. https://fakerjs.dev/about/announcements/2022-01-14.html. Accessed 20-Jul-2023

Sarker IH, Kayes A, Badsha S, Alqahtani H, Watters P, Ng A (2020) Cybersecurity data science: an overview from machine learning perspective. J Big data 7:1–29

Scalco S, Paramitha R, Vu DL, Massacci F (2022) On the feasibility of detecting injections in malicious npm packages. In: Proceedings of the 17th international conference on availability, reliability and security, pp 1–8

Sejfia A, Schäfer M (2022) Practical automated detection of malicious NPM packages. In: Proceedings of the 44th international conference on software engineering, pp 1681–1692

Shimada N, Xiao T, Hata H, Treude C, Matsumoto K (2022) Github sponsors: exploring a new way to contribute to open source. In: Proceedings of the 44th international conference on software engineering, pp 1058–1069

SSonatype (2021) State of the software supply chain report. https://www.sonatype.com/resources/state-of-the-software-supply-chain-2021

Viera AJ, Garrett JM (2005) Understanding interobserver agreement: the kappa statistic. Fam Med 37:360–363

Vu DL, Pashchenko I, Massacci F, Plate H, Sabetta A (2020) Towards using source code repositories to identify software supply chain attacks. In: Proceedings of the 2020 ACM SIGSAC conference on computer and communications security, pp 2093–2095

Wang D, Xiao T, Thongtanunam P, Kula RG, Matsumoto K (2021) Understanding shared links and their intentions to meet information needs in modern code review: a case study of the openstack and qt projects. Empir Softw Eng 26:1–32

Wang D, Xiao T, Son T, Kula RG, Ishio T, Kamei Y, Matsumoto K (2023) More than react: investigating the role of emoji reaction in github pull requests. Empir Softw Eng 28(5):123

Wattanakriengkrai S, Wang D, Kula RG, Treude C, Thongtanunam P, Ishio T, Matsumoto K (2022) Giving back: contributions congruent to library dependency changes in a software ecosystem. IEEE Trans Softw Eng 1–13. https://doi.org/10.1109/TSE.2022.3225197

Wittern E, Suter P, Rajagopalan S (2016) A look at the dynamics of the javascript package ecosystem. In: Proceedings of the 13th international conference on mining software repositories, pp 351–361

Zahan N, Zimmermann T, Godefroid P, Murphy B, Maddila C, Williams L (2022) What are weak links in the NPM supply chain? In: Proceedings of the 44th international conference on software engineering: software engineering in practice, pp 331–340

Zhou J, Wang S, Bezemer CP, Hassan AE (2020) Bounties on technical q&a sites: a case study of stack overflow bounties. Empir Softw Eng 25:139–177

Zhou J, Wang S, Bezemer CP, Zou Y, Hassan AE (2020) Studying the association between bountysource bounties and the issue-addressing likelihood of github issue reports. IEEE Trans Softw Eng 47(12):2919–2933

Zhou J, Wang S, Kamei Y, Hassan AE, Ubayashi N (2022) Studying donations and their expenses in open source projects: a case study of github projects collecting donations through open collectives. Empir Softw Eng 27:1–38

Zimmermann M, Staicu CA, Tenny C, Pradel M (2019) Small world with high risks: a study of security threats in the npm ecosystem. In: 28th USENIX security symposium (USENIX Security 19), pp 995–1010

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Youmei Fan is a Ph.D. student in Software Engineering Laboratory at the Graduate School of Science and Technology, Nara Institute of Science and Technology (NAIST). She completed a master's course in Software Engineering Laboratory at the Graduate School of Science and Technology, Nara Institute of Science and Technology (NAIST), Japan. During her master's degree, her main research interests related to the human aspect of software engineering, open-source software sustainability, and data mining. Find her at https://www.linkedin.com/in/youmei-fan-513a161a6/.

Dong Wang is currently an associate professor at Tianjin University, China. Prior to this, he worked as an assistant processor at Kyushu University, Japan. He received Ph.D. degree from Nara Institute of Science and Technology, Japan. He is a member of the IEEE. His research interests include mining software repositories, empirical software engineering, human aspects, and AI4SE. More about his information is available online at https://dong-w.github.io/.

Supatsara Wattanakriengkrai is a Ph.D. student in Software Engineering Laboratory at the Graduate School of Science and Technology, Nara Institute of Science and Technology (NAIST), Japan. She received a Master's degree from NAIST in 2021. Her main research interests include empirical software engineering, mining software repositories, and library dependencies in software ecosystems. More about her work is available online at https://www.supatsarawat.com/.

Hathaichanok Damrongsiri is a Master's student in the Software Engineering Laboratory at the Graduate School of Science and Technology, Nara Institute of Science and Technology (NAIST), Japan. She received a bachelor's degree in Software Engineering from Chiang Mai University and remains interested in the field of Software Engineering, especially in empirical software engineering, software educational materials, and teaching methods. Find her at https://www.linkedin.com/in/ploychanok/.

Christoph Treude is an Associate Professor at Singapore Management University (SMU). He serves on the editorial boards of journals including TSE, EMSE, and JSS, and will be PC Co-Chair for FSE 2026. His research goal – getting information to developers when and where they need it – is gaining new significance in the AI era. Before joining SMU, Christoph worked in Australia, Brazil, Canada, and Germany.

Hideaki Hata is an Associate Professor at Shinshu University. He received his Ph.D. in information science from Osaka University. His research interests include software ecosystems, human aspects in software engineering, and software economics. More about Hideaki and his work is available online at https://hideakihata.github.io/.

Raula Gaikovina Kula is a Professor at Osaka University. He received his Ph.D. from Nara Institute of Science and Technology (NAIST) in 2013, later joining as a Specially-Apppointed Assistant Professor from (2013-2016) at Osaka University. He then continued as a Specially-Appointed Assistant Professor from (2017), later becoming an Assistant Professor (2017-2023), and a Associate Professor (2023-2024) at NAIST. Kula has published over 150 publications in top Software Engineering venues, collaborating with several researchers from across the globe, and is a member of both Editorial and Steering Committees. Contact him at raula-k@ist.osaka-u.ac.jp.

Authors and Affiliations

Youmei Fan[1] · Dong Wang[2] · Supatsara Wattanakriengkrai[1] · Hathaichanok Damrongsiri[1] · Christoph Treude[3] · Hideaki Hata[4] · Raula Gaikovina Kula[5]

✉ Youmei Fan
fan.youmei.fs2@is.naist.jp

Dong Wang
dong_w@tju.edu.cn

Supatsara Wattanakriengkrai
wattanakri.supatsara.ws3@is.naist.jp

Hathaichanok Damrongsiri
damrongsiri.hathaichanok.db5@is.naist.jp

Christoph Treude
ctreude@smu.edu.sg

Hideaki Hata
hata@shinshu-u.ac.jp

Raula Gaikovina Kula
raula-k@ist.osaka-u.ac.jp

1 Nara Institute of Science and Technology, Ikoma, Japan

2 College of Intelligence and Computing, Tianjin University, Tianjin, China

3 Singapore Management University, Singapore, Singapore

4 Shinshu University, Matsumoto, Japan

5 Osaka University, Suita, Japan